Práctica 14 — Monedas que aparecen solas

Programación de Videojuegos · FP Lumara · Práctica previa: 08

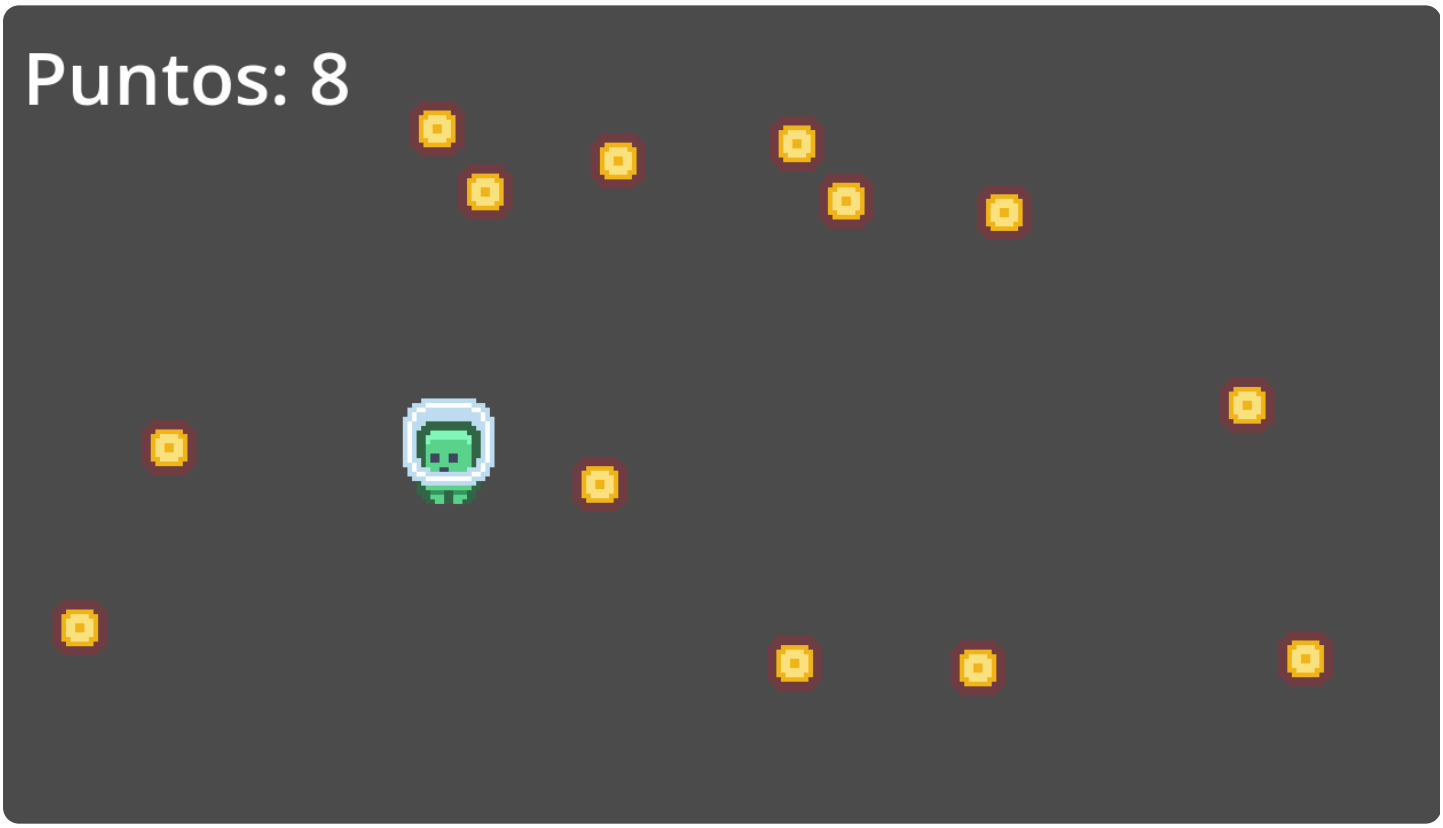
> *Lo que aprendes hoy: a crear objetos desde el código mientras el juego se está jugando.*

---

1. Qué vas a conseguir

Cada segundo y medio aparece una moneda nueva en un punto **al azar** de la pantalla. Las recoges y el marcador sube. No se acaban nunca.

[CAPTURA 14.1]



"El resultado de la práctica 14: las monedas aparecen automáticamente."

---

2. El problema

Hasta ahora, todo lo que aparecía en tus juegos estaba **colocado a mano en el editor**. Las tres monedas de la práctica 08, las cuatro cajas de la 09, los murciélagos de la 13.

Eso sirve para un escenario fijo, pero no para:

- Enemigos que llegan por oleadas.
- Disparos, que se crean al pulsar.
- Monedas que salen cada cierto tiempo.

Ninguna de esas cosas se puede poner de antemano, porque **no se sabe cuántas va a haber**. Hay que crearlas mientras el juego corre.

---

3. Materiales

Descomprime `practica_14_inicio.zip`. Es tu juego de la práctica 08 **sin las tres monedas**: están el personaje, el marcador y el archivo `escenas/moneda.tscn`, pero la pantalla arranca vacía.

SESIÓN 1 — Crear una moneda desde el código

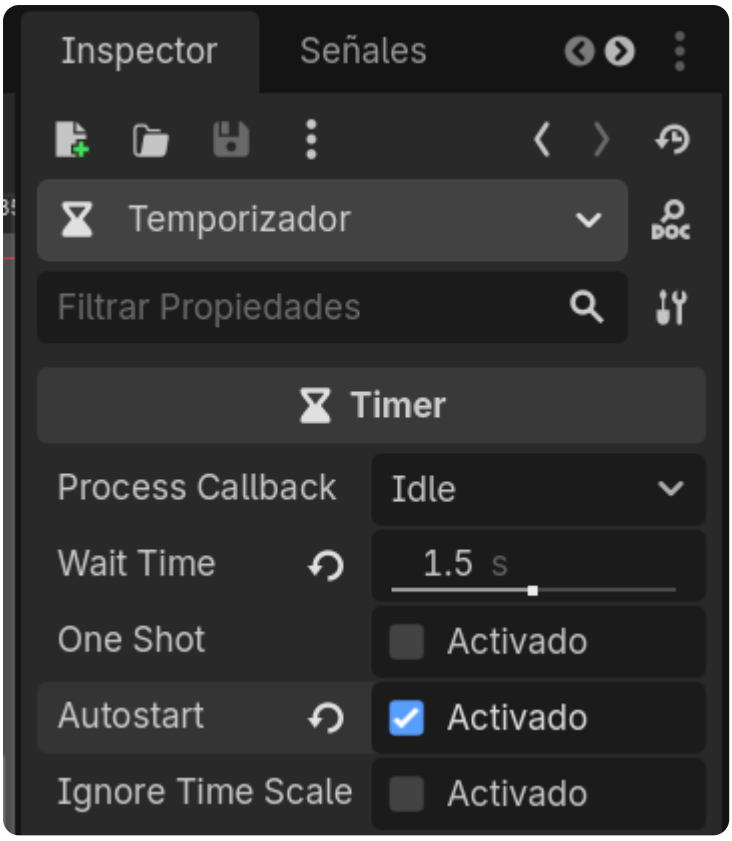
---

4. El temporizador

Necesitamos algo que avise cada cierto tiempo. Godot tiene un nodo para eso.

1. Selecciona `Mundo` y añade un hijo de tipo `Timer`. Llámalo `Temporizador`.
2. En el Inspector, pon **Tiempo de Espera** a `1.5` segundos.
3. Marca la casilla **Inicio Automático**, para que arranque solo al empezar el juego.

[CAPTURA 14.2]



"Un temporizador que se activa cada 1,5 segundos."

4. Conecta su señal: con `Temporizador` seleccionado, panel **Nodo**, pestaña **Señales**, doble clic en `timeout`. Conéctala a `Mundo` con el método `_on_temporizador_timeout`.

---

5. Crear la moneda

5. Abre `scripts/mundo.gd` y añade **arriba del todo**, debajo del `extends`:

```
const ESCENA_MONEDA := preload("res://escenas/moneda.tscn")
```

6. Y completa la función que te ha creado Godot:

```
func _on_temporizador_timeout() -> void:
    var moneda := ESCENA_MONEDA.instantiate()
    moneda.position = Vector2(190, 90)
    add_child(moneda)
```

7. Ejecuta con **F6**. Cada segundo y medio aparece una moneda en el centro. Fíjate que, con lo que hemos hecho hasta ahora, salen todas en el mismo sitio y no suman puntos.

6. Posición aleatoria

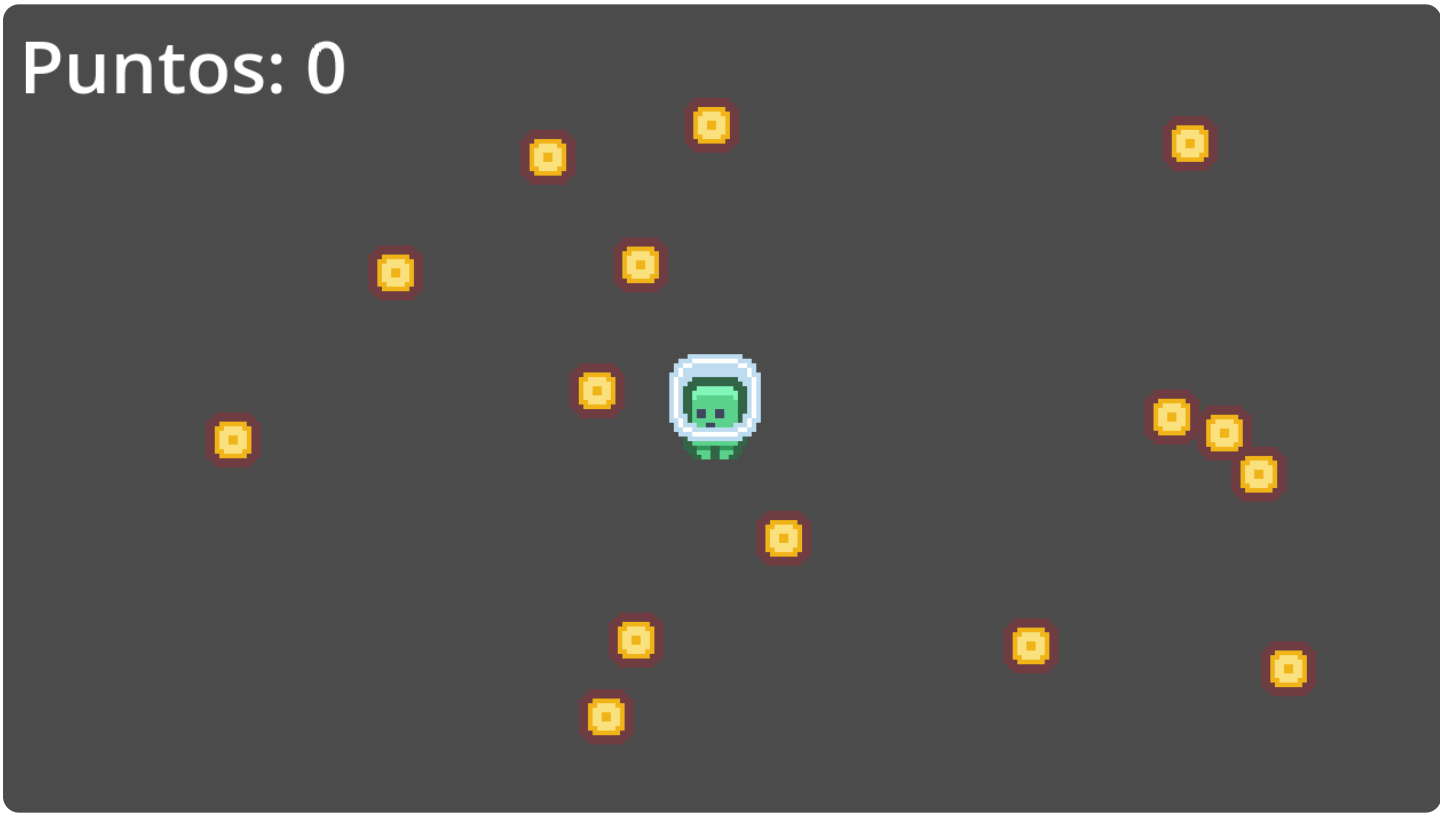
8. Cambia la línea de la posición por esta:

```
    moneda.position = Vector2(randf_range(20, 300), randf_range(20, 160))
```

9. Ejecuta. Ahora cada moneda aparece en un sitio distinto.

Hemos determinado el rango X entre 20 y 30, y el rango Y entre 20 y 160. Como la pantalla mide **320 x 180**, dejamos 20 píxeles de margen por cada lado para que ninguna moneda salga cortada por el borde.

[CAPTURA 14.3]



“randf_range devuelve un número al azar dentro de un intervalo.”

7. Que sumen puntos

Si pruebas a recoger una moneda, desaparece pero el marcador no sube. ¿Por qué?

Porque en la práctica 08 conectabas la señal `recogida` **a mano, en el editor**. Estas monedas **no existen en el editor**: nacen mientras el juego corre. Así que la conexión también hay que hacerla desde el código.

10. Añade una línea a la función:

```
func _on_temporizador_timeout() -> void:
    var moneda := ESCENA_MONEDA.instantiate()
    moneda.position = Vector2(randf_range(20, 300), randf_range(20, 160))
    moneda.recogida.connect(_on_moneda_recogida)
    add_child(moneda)
```

11. Ejecuta y comprueba que ahora sí suman.

El script completo queda así:

```
extends Node2D

const ESCENA_MONEDA := preload("res://escenas/moneda.tscn")

var puntos := 0

@onready var marcador: Label = $Interfaz/Puntos


func _ready() -> void:
    actualizar_marcador()


func _on_temporizador_timeout() -> void:
    var moneda := ESCENA_MONEDA.instantiate()
    moneda.position = Vector2(randf_range(20, 300), randf_range(20, 160))
    moneda.recogida.connect(_on_moneda_recogida)
    add_child(moneda)


func _on_moneda_recogida() -> void:
    puntos += 1
    actualizar_marcador()


func actualizar_marcador() -> void:
    marcador.text = "Puntos: " + str(puntos)
```

8. El código, línea a línea

```
const ESCENA_MONEDA := preload("res://escenas/moneda.tscn")
```

Tres cosas nuevas de golpe:

- `const` es como `var`, pero el valor **no se puede cambiar** después. Por eso se escribe en mayúsculas, por costumbre: así se distingue de un vistazo.
- `preload` carga el archivo **al abrir el juego**, no cuando hace falta. Como la moneda se va a crear muchas veces, conviene tenerla lista y no ir al disco cada vez.
- Lo que guarda es una **escena empaquetada**: el molde de la práctica 08, ahora metido en una variable.

`ESCENA_MONEDA.instantiate()` Fabrica **una copia** del molde. Es exactamente lo mismo que arrastrar el archivo al panel Escena, pero hecho desde el código.

Ojo: la copia existe, pero **todavía no está en el juego**. Es como tener un mueble montado en el pasillo.

`add_child(moneda)` Mete la moneda dentro del árbol de nodos, colgando de `Mundo`. A partir de esta línea la moneda existe de verdad: se dibuja, detecta colisiones y hace lo que tenga que hacer.

Sin `add_child`, la moneda no aparece. Es el error número uno de la práctica.

`randf_range(20, 300)` Devuelve un número con decimales al azar entre 20 y 300. Existe también `randi_range()` si quieres números enteros.

`moneda.recogida.connect(_on_moneda_recogida)` Conectar una señal desde el código. Fíjate en la forma: `nodo.señal.connect(función)`.

La función va **sin paréntesis**. No la estás llamando: le estás diciendo a la señal cuál tiene que llamar cuando llegue el momento.

> ***Esta línea es la que separa la práctica 08 de la 14. Lo que antes hacías arrastrando el ratón en el editor, ahora lo hace una línea de código, porque el editor no puede conectar algo que todavía no existe.***

---

9. Pruébalo

- [ ] Aparece una moneda cada segundo y medio.
- [ ] Cada una en un sitio distinto.

- [ ] Ninguna sale cortada por el borde de la pantalla.
- [ ] Al recogerlas, el marcador sube.
- [ ] Si dejas el juego corriendo un minuto, la pantalla se llena de monedas.
- [ ] En el panel **Salida** no hay nada en rojo.

---

10. La teoría en cinco líneas

- Una escena guardada se puede **instanciar desde el código** con `instantiate()`.
- La copia no existe en el juego hasta que se mete en el árbol con `add_child()`.
- `preload` carga el archivo una vez, al arrancar.
- `randf_range()` da un número al azar dentro de un intervalo.
- Lo que nace durante la partida hay que **conectarlo por código**, porque el editor no puede.

---

11. Si no te funciona

Lo que ves	Qué pasa	Cómo se arregla
No aparece ninguna moneda	Falta `add_child()`	Es la línea que la mete en el juego
Tampoco aparece ninguna	El Timer no tiene Inicio Automático	Marca la casilla
Aparece solo una y ya está	El Timer no es repetitivo, o han desmarcado algo	Comprueba el Inspector
Salen todas en el mismo sitio	Sigue la posición fija	Cambia por `randf_range`
Salen cortadas por el borde	Los márgenes son muy justos	Usa de 20 a 300 y de 20 a 160
Desaparecen pero no suman	Falta la línea del `connect`	Es la del punto 10
`Invalid call ... 'connect'`	Han puesto paréntesis en la función	Va sin ellos
Todas salen en el mismo sitio siempre que ejecutas	Se están fijando en el primer valor	Cada copia calcula el suyo, revisa dónde pusiste la línea
El juego se ralentiza mucho	Llevan cinco minutos y hay 200 monedas	Es real: mira el reto amarillo

---

12. Retos

Verde · Cambia el **Tiempo de Espera** del temporizador a `0.3` y ejecuta. Después a `5`. Quédate con el ritmo que te parezca más jugable.

Amarillo · Haz que las monedas **desaparezcan solas** si nadie las recoge en cinco segundos. Pista: cada moneda puede tener su propio `Timer` dentro, o puedes investigar el método `get_tree().create_timer()`.

Rojo · Que no pueda haber **más de diez monedas a la vez** en la pantalla. Pista: necesitas saber cuántas hay antes de crear otra. Ya viste en el Hito 1 cómo contar los nodos de un grupo.

---

13. Autoevaluación

- [ ] Sé montar un `Timer` y conectar su señal.
- [ ] Sé qué hace `preload` y por qué se usa aquí.
- [ ] Sé instanciar una escena desde el código.
- [ ] Sé que sin `add_child()` la copia no aparece.
- [ ] Sé generar números al azar dentro de un intervalo.
- [ ] Sé conectar una señal desde el código.
- [ ] Sé explicar por qué aquí no se puede conectar desde el editor.

---

14. Vocabulario

Español	Inglés	Qué es
Instanciar	Instantiate	Fabricar una copia de una escena
Escena empaquetada	PackedScene	El molde guardado en un archivo
Precargar	Preload	Cargar al arrancar, no cuando hace falta
Constante	Const	Variable cuyo valor no cambia
Temporizador	Timer	Nodo que avisa cada cierto tiempo
Aleatorio	Random	Al azar

---

15. Entrega

Comprime la carpeta como `apellido_practica14.zip` y súbela al Aula Virtual de Alexia, al final de la **segunda** sesión.

Capturas necesarias en esta práctica

Código	Contenido
14.1	Resultado final, a ser posible en GIF
14.2	Inspector del Timer con Tiempo de Espera e Inicio Automático
14.3	Pantalla con muchas monedas repartidas al azar